

Лекція 3. Поняття системи контролю версій у контексті неперервної інтеграції

У процесі неперервної інтеграції система контролю версій виконує кілька важливих функцій, які сприяють автоматизації, ефективності та узгодженості роботи команди розробників. Однією з таких функцій є централізація змін. Код зберігається в єдиному репозиторії, який є доступним для всіх учасників проєкту. Це забезпечує кожного розробника актуальною версією коду, запобігаючи дублюванню та втраті даних.

Відстеження історії змін є ще однією важливою складовою. Система дозволяє зберігати всі внесені коміти з інформацією про те, хто та коли вніс певні зміни. Це полегшує аналіз еволюції проєкту та сприяє швидкому виявленню причин потенційних проблем у коді.

Система контролю версій також є основою для автоматизації перевірок. Будь-які зміни в коді запускають процеси автоматизованого тестування, збірки та перевірки якості коду. Це дозволяє вчасно виявляти помилки та забезпечувати стабільність розробки. Така інтеграція з CI/CD-пайплайнами підвищує надійність програмного продукту.

Підтримка колаборації є невід'ємною перевагою системи контролю версій. Завдяки використанню гілок (branches) розробники можуть паралельно працювати над різними функціями, не створюючи конфліктів у коді. Цей підхід дозволяє гнучко розподіляти задачі між членами команди та впроваджувати нові функціональні можливості без ризику порушення основного функціоналу.

Останнім елементом є інтеграція, яка забезпечує можливість швидкого злиття змін із різних гілок у головну гілку репозиторію. Це гарантує актуальність основної версії коду та дозволяє швидко підготувати його для релізу. Також така інтеграція сприяє мінімізації конфліктів під час об'єднання змін.

Система контролю версій є важливим інструментом у процесі неперервної інтеграції, оскільки вона забезпечує ефективне управління змінами, сприяє автоматизації та підтримує колективну роботу розробників.

Порівняння централізованих і розподілених систем контролю версій

Характеристика	Централізовані системи (CVCS)	Розподілені системи (DVCS)
Архітектура	Єдиний центральний сервер, копії на локальних пристроях	Кожен розробник має повну копію репозиторія
Приклади систем	Subversion (SVN), Perforce	Git, Mercurial
Доступ до історії змін	Лише через центральний сервер	Локальний доступ до повної історії змін
Робота без підключення до сервера	Неможлива	Можлива (локальні операції)
Швидкість операцій	Залежить від швидкості з'єднання	Швидші локальні операції
Можливість роботи з гілками	Підтримка є, але обмежена	Потужна підтримка гілок і злиттів
Масштабованість	Знижується з розміром команди	Підходить для великих команд і проектів
Приклади інструментів для CI	Jenkins, TeamCity (без прямої інтеграції)	GitHub Actions, GitLab CI/CD, Jenkins (повна інтеграція)

Централізовані системи, такі як Subversion (SVN) та Perforce, зберігають всі дані на одному центральному сервері, що дозволяє спрощувати адміністрування та контроль доступу. Однак вони мають певні обмеження, зокрема, потребують постійного з'єднання з сервером для доступу до історії змін та виконання комітів.

Розподілені системи контролю версій, як Git і Mercurial, зберігають повні копії репозиторіїв на локальних пристроях розробників, що дозволяє працювати в автономному режимі, зменшуючи залежність від центрального сервера. Вони також підтримують потужні механізми для роботи з гілками та злиттям змін, що є критично важливим для великих команд та проектів. Проте такі системи можуть вимагати більше місця на диску та бути складнішими для новачків через велику кількість доступних функцій.

Централізовані системи контролю версій (CVCS)

Централізовані системи контролю версій (Centralized Version Control Systems, CVCS) є одним із традиційних підходів до управління змінами в програмному забезпеченні, який широко використовувався до появи розподілених систем. У цій архітектурі всі дані проєкту, включаючи файли, історію змін та метадані, зберігаються на центральному сервері. Розробники отримують локальні копії потрібних файлів із сервера, вносять зміни та завантажують їх назад до центрального репозиторію. Центральний сервер виконує функцію єдиної точки синхронізації, де всі члени команди можуть взаємодіяти з актуальними даними.

Особливістю централізованих систем є їх організованість і централізація управління. Завдяки цьому розробники завжди працюють з однією версією репозиторію, що дозволяє уникати дублювання змін і підтримувати єдину структуру проєкту. Також централізована модель добре підходить для організацій із чітко визначеною ієрархією доступу та адміністративними вимогами.

Серед популярних прикладів централізованих систем можна виділити Subversion (SVN) та Perforce. Ці інструменти забезпечують базовий функціонал для управління версіями, включаючи можливість відслідковувати зміни, створювати резервні копії та відновлювати попередні версії файлів. Вони стали популярними завдяки простоті впровадження і зрозумілості процесу управління даними.

Розподілені системи контролю версій (DVCS)

Розподілені системи контролю версій (Distributed Version Control Systems, DVCS) представляють сучасний підхід до управління змінами в програмному забезпеченні, який вирізняється високою гнучкістю, автономністю та продуктивністю. Основною особливістю таких систем є те, що кожен розробник має повну копію репозиторія, яка включає не лише актуальну версію файлів, а й всю історію змін, метадані та гілки. Завдяки цьому архітектурному підходу

розробники можуть працювати автономно, не залежачи від доступу до центрального сервера. Вони можуть переглядати попередні версії файлів, створювати коміти, зливати гілки та вирішувати конфлікти, навіть перебуваючи в умовах відсутності інтернет-з'єднання.

Одними з найбільш популярних прикладів розподілених систем є Git і Mercurial. Git, створений Лінусом Торвальдсом у 2005 році для розробки ядра Linux, став стандартом у багатьох галузях завдяки своїй швидкості, масштабованості та багатому набору функцій. Він є основою для таких популярних платформ, як GitHub, GitLab і Bitbucket, які забезпечують додаткові можливості для командної співпраці, включаючи управління репозиторіями, автоматизацію процесів CI/CD та інструменти для код-рев'ю.

Функціональність розподілених систем дозволяє організувати ефективну роботу як у невеликих командах, так і у великих міжнародних організаціях. Завдяки підтримці складних робочих процесів, таких як паралельна розробка різних функціональностей у гілках, вони забезпечують ефективність навіть у випадках, коли над одним проєктом працює декілька сотень чи тисяч розробників.

Важливою перевагою DVCS є здатність забезпечувати швидкодію завдяки локальним операціям. Наприклад, операції з історією комітів, створення гілок або пошуку змін виконуються локально на пристрої розробника, що дозволяє уникати затримок, пов'язаних із мережею. Це не лише підвищує продуктивність роботи, але й сприяє кращій організації складних процесів розробки, таких як інтеграція та тестування змін.

Окрім цього, DVCS пропонують підвищену стійкість до збоїв. Якщо центральний сервер тимчасово недоступний, команда може продовжувати працювати автономно, а всі зміни будуть синхронізовані з сервером, як тільки доступ буде відновлено. Це значно знижує ризик простоїв і втрати даних, що є критично важливим для великих команд і проєктів із високими вимогами до безперервної розробки.

Попри свої численні переваги, DVCS мають і певні особливості, які варто враховувати. Зокрема, кожен розробник має зберігати повну копію репозиторія, що може створити підвищені вимоги до дискового простору, особливо у великих проєктах із довгою історією змін. Наприклад, великі репозиторії з десятками гігабайтів даних можуть вимагати значного місця на пристроях розробників.

Ще одним викликом є більш крута крива навчання для новачків. Інструменти на основі DVCS, такі як Git, мають широкий функціонал і підтримують складні робочі сценарії, але це може спантеличувати нових користувачів. Наприклад, процеси вирішення конфліктів при злитті гілок або робота з віддаленими репозиторіями можуть здаватися складними без належної підготовки. Однак ця проблема вирішується завдяки розвиненій документації, навчальним матеріалам і підтримці великої спільноти користувачів.

Git

Git був створений у 2005 році Лінусом Торвальдсом, розробником ядра операційної системи Linux. Основна мотивація створення полягала в необхідності створити власну систему контролю версій після завершення використання комерційної системи BitKeeper, яку Linux-спільнота використовувала раніше.



GIT

Вийшов із спільноти розробників Linux

Засновник: Лінус Торвальдс, 2005

Початкові цілі:

- швидкість
- Підтримка нелінійної розробки (тисячі паралельних гілок)
- Повністю роздано
- Здатний ефективно керувати великими проєктами, такими як Linux
- Глобальний інформаційний трекер (Global Information Tracker)

Перша версія Git була випущена у квітні 2005 року, і з того часу він став основною системою контролю версій для більшості програмних проєктів у світі.

Головна мета Git – забезпечити ефективний і надійний інструмент для керування версіями програмного забезпечення.

Основні командами Git та їх поясненням:

Команда	Призначення	Приклад
git init	Ініціалізує новий локальний репозиторій	git init
git clone <url>	Клонує існуючий репозиторій із сервера в локальний каталог	git clone https://github.com/user/repo.git
git status	Показує поточний статус репозиторію, включаючи змінені файли та файли, які ще не додані до індексації	git status
git add <file>	Додає файл до індексації (staging area) перед створенням коміту	git add file.txt
git commit -m "<msg>"	Створює коміт із доданими файлами з повідомленням, що описує зміни	git commit -m "Added new feature"
git log	Відображає історію комітів репозиторію	git log
git branch	Виводить список локальних гілок або створює нову гілку	git branch feature-branch
git checkout <branch>	Перемикається на зазначену гілку або відновлює файли з певної версії	git checkout feature-branch
git switch <branch>	Перемикається на іншу гілку (сучасна альтернатива git checkout)	git switch develop
git merge <branch>	Зливає зміни з однієї гілки в поточну	git merge feature-branch
git pull	Завантажує останні зміни з віддаленого репозиторію та автоматично зливає їх у поточну гілку	git pull origin main
git push	Завантажує локальні зміни у віддалений репозиторій	git push origin main
git remote -v	Показує список віддалених репозиторіїв та їх URL-адреси	git remote -v
git fetch	Завантажує останні зміни з віддаленого репозиторію, не зливаючи їх із локальними змінами	git fetch origin
git diff	Показує різницю між файлами в різних версіях (наприклад, робоча директорія vs staging area)	git diff
git stash	Зберігає незавершені зміни у тимчасове сховище, дозволяючи	git stash

	перемикається між гілками без втрати змін	
git stash apply	Відновлює зміни з тимчасового сховища (stash)	git stash apply
git reset <file>	Видаляє файл із staging area, не видаляючи його з робочої директорії	git reset file.txt
git reset --hard <commit>	Повертає репозиторій до певного коміту, видаляючи всі подальші зміни	git reset --hard HEAD~1
git revert <commit>	Створює новий коміт, що скасовує зміни, внесені в зазначеному коміті	git revert abc1234
git tag <name>	Додає тег до конкретного коміту (зазвичай для маркування релізів)	git tag v1.0.0
git rm <file>	Видаляє файл із робочої директорії та staging area	git rm file.txt
git blame <file>	Показує, хто і коли вніс зміни у кожен рядок файлу	git blame file.txt
git show <commit>	Відображає детальну інформацію про конкретний коміт	git show abc1234
git config	Налаштовує параметри Git, наприклад, ім'я користувача чи email	git config --global user.name "John"
git cherry-pick <commit>	Переносить зміни з одного коміту в іншу гілку	git cherry-pick abc1234
git clean -f	Видаляє незбережені файли або директорії з робочої директорії	git clean -f

Ця таблиця охоплює основні команди Git, які використовуються для керування версіями, гілками та співпраці в проєктах.

Роль систем контролю версій у неперервній інтеграції

Роль систем контролю версій у неперервній інтеграції (Continuous Integration, CI) є ключовою, оскільки вони забезпечують ефективне управління змінами коду, їх інтеграцію та перевірку. У системах CI найпопулярнішою є розподілена модель контролю версій, зокрема Git, завдяки її гнучкості, високій швидкодії та широким можливостям інтеграції з платформами на кшталт GitHub, GitLab чи Bitbucket.

Розподілені системи контролю версій дозволяють організувати процес розробки таким чином, щоб максимально полегшити впровадження практик CI. Однією з найважливіших функцій є можливість інтеграції змін у кодову базу з автоматизованою перевіркою якості через скрипти CI. Такі інструменти, як

Jenkins, GitHub Actions, GitLab CI/CD або CircleCI, автоматично запускають тести, збірку, аналіз коду та інші перевірки при внесенні змін у репозиторій. Це забезпечує швидке виявлення помилок, підвищуючи загальну якість програмного забезпечення.

Використання гілок у розподілених системах дозволяє ефективно організувати робочий процес. Розробники можуть створювати окремі гілки для кожної нової функції, виправлення помилки чи експерименту. Це ізолює зміни, мінімізуючи ризики конфліктів у кодовій базі. Завдяки механізмам Git, таким як pull requests або merge requests, можна інтегрувати зміни в основну гілку лише після їх перевірки та схвалення. Це допомагає підтримувати стабільність основної версії проєкту.

Ще однією важливою перевагою є можливість швидкого вирішення конфліктів. У разі виникнення конфліктів у коді Git надає потужні інструменти для їх локалізації та усунення. Крім того, система дозволяє переглядати історію змін, аналізувати, які саме зміни спричинили конфлікт, і швидко приймати рішення щодо їх вирішення. Це дозволяє командам ефективно підтримувати стабільність головної гілки (main або master), яка часто використовується для релізів.

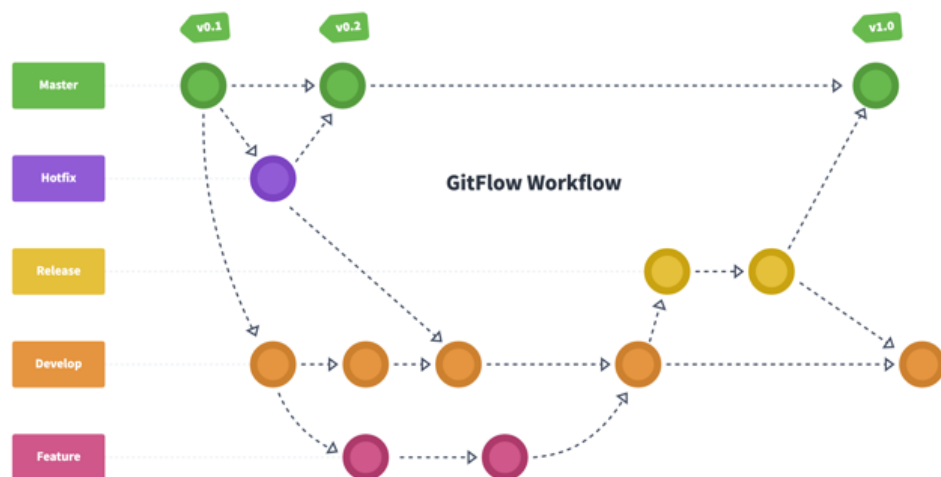
Інтеграція розподілених систем із CI також сприяє автоматизації управління релізами. Наприклад, платформи на основі Git дозволяють створювати теговані релізи, що запускають автоматизовані процеси збірки, тестування та розгортання в продуктивне середовище. Це забезпечує швидший цикл випуску нових версій продукту та зменшує людський фактор у процесі розробки.

Таким чином, розподілені системи контролю версій, особливо Git, є незамінним елементом у реалізації CI. Вони забезпечують високу гнучкість, ефективне управління змінами та стабільність кодової бази, що дозволяє командам швидко інтегрувати, перевіряти та доставляти якісний програмний продукт.

3.3. Поняття Git Flow. Типи Git Flow

Git Flow – це одна з популярних моделей робочих процесів у системі контролю версій Git, яка пропонує чітке визначення того, як має відбуватися робота з гілками у проєкті. Ця модель була розроблена Vincent Driessen у 2010 році і стала стандартом для багатьох команд завдяки своїй структурованості та здатності впорядковувати складні процеси розробки, тестування та релізу програмного забезпечення. Вона особливо ефективна для командної роботи над великими проєктами, які потребують частих оновлень і чіткої організації.

GIT Flow



Головна мета Git Flow – це оптимізація та впорядкування таких етапів розробки:

- *розробка нових функцій*: кожна функція створюється та розвивається окремо, без впливу на стабільну версію програми;
- *тестування і стабілізація*: спеціальні гілки дозволяють проводити повне тестування перед фінальним релізом;
- *реліз та підтримка*: забезпечується стабільність основної гілки, яка відповідає за продакшн-версію, та можливість швидко виправляти критичні помилки.

Основні типи Git Flow

1) Основні гілки:

- **master** (або **main**): Ця гілка містить стабільний код, готовий до випуску або вже випущений в продукцію. Всі зміни, які потрапляють до цієї гілки, повинні бути перевірені і протестовані.

Зазвичай використовують теги для позначення версій:

git tag -a v1.0 -m "Реліз версії 1.0"

- **develop**: гілка для інтеграції нових функцій, яка представляє собою активну розробку. Усі зміни, що додаються до цієї гілки, тестуються, і після стабілізації та завершення розробки нових функцій код зливається в master для випуску.

2) Допоміжні гілки:

- **feature** (особливі гілки для розробки нових функцій): вони створюються від гілки develop і використовуються для роботи над новими можливостями або змінами. Кожна нова функція розробляється в окремій гілці feature, що дозволяє ізолювати її від основної розробки. Після завершення розробки функції, її зливають з гілкою develop.
- **release** (гілки для підготовки до релізу): коли нові функції, що розробляються в гілці develop, готові до релізу, створюється гілка release. Вона служить для стабілізації коду перед випуском, включаючи виправлення багів, оновлення документації та проведення фінальних тестів. Після завершення роботи, зміни з гілки release зливаються з master і develop.
- **hotfix** (гарячі виправлення): гілки для термінових виправлень у коді, які потрібно випустити без очікування стабільної версії. Вони створюються безпосередньо від гілки master, коли виникає потреба в терміновому виправленні помилки в продукції. Після виправлення багу, зміни з гілки hotfix зливаються в master (для публікації виправлення) і develop (для збереження актуальності основної гілки розробки).

Алгоритм роботи з Git Flow:

○ створення нової гілки *feature*:

- коли розпочинається робота над новою функцією, створюється окрема гілка *feature* на основі гілки *develop*;
- назва гілки повинна бути описовою, щоб зрозуміло, над якою функцією ведеться робота (наприклад, *feature/login-system*)

git checkout develop

git checkout -b feature/<назва_функції>

- усі зміни, що стосуються цієї функції, зберігаються у гілці *feature*.

○ злиття гілки *feature* із *develop*:

- після завершення роботи над функцією і перевірки її працездатності, гілка *feature* зливається назад у *develop*;
- перед злиттям бажано оновити гілку *feature*, синхронізувавши її з актуальним станом *develop*:

git checkout feature/<назва_функції>

git pull origin develop

- Після цього виконується злиття:

git checkout develop

git merge feature/<назва_функції>

- Якщо гілка більше не потрібна, її можна видалити:

git branch -d feature/<назва_функції>

○ створення гілки *release* для стабілізації:

- коли в гілці *develop* є готовий набір функцій, створюється гілка *release* для підготовки релізу;
- у цій гілці відбувається фінальне тестування, дрібні виправлення та підготовка до релізу (наприклад, оновлення документації, конфігурацій тощо):

git checkout develop

git checkout -b release/<номер_релізу>

- усі виправлення додаються в гілці *release*;

○ злиття гілки *release* у *master* та *develop*:

- після завершення роботи над стабілізацією коду гілка *release* зливається в *master* для випуску нової версії:

git checkout master

git merge release/<номер_релізу>

- для збереження хронології розробки ці ж зміни інтегруються в *develop*:

git checkout develop

git merge release/<номер_релізу>

- гілка *release* більше не потрібна і може бути видалена:

git branch -d release/<номер_релізу>

- позначається нова версія у *master*:

git tag -a v<номер_версії> -m "Опис релізу"

○ робота з гілками *hotfix* у разі критичних помилок:

- якщо виявлено критичну помилку у продукційній версії, створюється гілка *hotfix* від *master* для оперативного виправлення.

git checkout master

git checkout -b hotfix/<назва_виправлення>

- після внесення змін гілка *hotfix* зливається в *master* для негайного оновлення продукційної версії:

git checkout master

git merge hotfix/<назва_виправлення>

- зміни також інтегруються у *develop*, щоб вони не були втрачені в наступних релізах:

git checkout develop

git merge hotfix/<назва_виправлення>

- гілка *hotfix* видаляється після завершення роботи:

git branch -d hotfix/<назва_виправлення>

Додаткові рекомендації:

- *назви гілок*: використовуйте зрозумілі й стандартизовані назви для гілок (feature/, release/, hotfix/), щоб у команді не виникало плутанини;
- *оновлення гілок*: регулярно синхронізуйте гілки з віддаленим репозиторієм, щоб уникнути конфліктів:

git pull origin develop

- *рев'ю коду*: перед злиттям гілок, особливо feature чи hotfix, обов'язково проводьте код-рев'ю, щоб гарантувати якість змін.

Git Flow – це потужна, але складна модель, яка підходить для проєктів середнього та великого розміру з довготривалим циклом розробки. Для менших проєктів або команд із динамічним стилем роботи вона може бути занадто складною і неефективною. Перед впровадженням Git Flow варто оцінити потреби проєкту та команди, щоб уникнути зайвих витрат часу та ресурсів.